

INSTITUTO POLITECNICO NACIONAL

UPIITA - Unidad Profesional Interdisciplinaria de Ingenieria
en Tecnologias Avanzadas

Aplicaciones Distribuidas

Reporte de Practica

Conexión y Consulta a MongoDB Atlas con Node.js

Alumno: Daniel Galicia

1. Introduccion

En esta practica se implemento una aplicacion en Node.js que permite conectarse a un cluster de MongoDB Atlas, una base de datos NoSQL alojada en la nube. El objetivo principal fue aprender a realizar consultas sobre colecciones de datos reales utilizando el driver oficial de MongoDB para Node.js.

MongoDB es una base de datos orientada a documentos, lo que significa que los datos se almacenan en formato similar a JSON (llamado BSON), permitiendo estructuras flexibles y consultas eficientes. MongoDB Atlas es la version administrada en la nube que ofrece MongoDB, facilitando el despliegue y gestion del servidor sin necesidad de instalacion local.

Para esta practica se utilizo la base de datos de ejemplo sample_mflix, que contiene informacion sobre peliculas, comentarios y otros datos relacionados con el cine. Esta base de datos es provista por MongoDB Atlas para que los usuarios puedan practicar consultas sin necesidad de crear sus propios conjuntos de datos.

2. Objetivo

Establecer una conexion exitosa con un cluster de MongoDB Atlas desde una aplicacion Node.js, y realizar consultas sobre las colecciones comments y embedded_movies de la base de datos sample_mflix, mostrando los resultados en la consola de manera ordenada.

3. Desarrollo

3.1 Configuracion del entorno

Para comenzar se instalo el driver oficial de MongoDB para Node.js mediante el gestor de paquetes npm. Tambien se importo el modulo dns de Node.js para configurar manualmente los servidores DNS de Google (8.8.8.8 y 8.8.4.4), ya que sin esta configuracion el sistema no podia resolver el nombre del servidor de MongoDB Atlas y arrojaba un error ECONNREFUSED.

Las dependencias utilizadas en el proyecto fueron:

- mongodb: driver oficial para Node.js
- dns (nativo de Node.js): para configurar los servidores DNS
- assert (nativo de Node.js): para verificacion de condiciones

3.2 Cadena de conexion

Se utilizo el URI de conexion proporcionado por MongoDB Atlas, el cual incluye el nombre de usuario, la contraseña y el nombre del cluster. Esta cadena permite que el cliente de MongoDB sepa a que servidor conectarse:

```
const uri =  
"mongodb+srv://DanielG:DanielG@cluster0.kyxjy2h.mongodb.net/?appName=Cluster0";  
const client = new MongoClient(uri);
```

El protocolo mongodb+srv indica que se usa un esquema de conexion con SRV DNS, lo que permite que el driver resuelva automaticamente los hosts del replica set de MongoDB Atlas.

3.3 Funciones implementadas

Se definieron cuatro funciones asincronas principales, cada una con una responsabilidad especifica:

iterateFunc(doc)

Funcion auxiliar que recibe un documento y lo imprime en la consola con formato JSON legible (indentacion de 4 espacios).

listDatabases(client)

Conecta con el servidor usando privilegios de administrador y solicita la lista de todas las bases de datos disponibles en el cluster. Imprime los nombres de cada base de datos encontrada.

findAllData(client)

Realiza una consulta sobre la coleccion movies de la base de datos sample_mflix. Usa find({}) para traer todos los documentos pero con un limit(2) para obtener solo los primeros dos resultados. Imprime el titulo de la primera pelicula encontrada y muestra los documentos completos en formato JSON.

findComments(client)

Consulta la coleccion comments de sample_mflix sin ningun filtro y con un limite de 5 documentos. Convierte el cursor resultante en un arreglo con toArray() y lo imprime en consola.

findEmbeddedMovies(client)

Consulta la coleccion `embedded_movies` de `sample_mflix`, tambien sin filtros y con limite de 5 documentos. Esta coleccion es especial porque cada pelicula incluye un campo `plot_embedding` que contiene un arreglo de valores numericos (vectores de embeddings), lo que la hace mas pesada en datos.

3.4 Funcion principal (main)

La funcion `main` es la que orquesta la ejecucion del programa. Primero crea el cliente de MongoDB con la URI de conexion, luego llama a `client.connect()` para establecer la conexion, y finalmente ejecuta las funciones de consulta. El bloque `try-catch-finally` garantiza que si ocurre algun error se muestre en consola, y que la conexion siempre se cierre al terminar mediante `client.close()`.

Para esta ejecucion se dejaron comentadas las funciones `listDatabases` y `findAllData`, y se ejecutaron unicamente:

- `findComments(client)`
- `findEmbeddedMovies(client)`

4. Resultados

4.1 Comentarios obtenidos (coleccion: comments)

Se recuperaron exitosamente 5 documentos de la coleccion comments. Cada documento contiene los campos: _id, name, email, movie_id, text y date. A continuacion se resumen los datos obtenidos:

Nombre	Email	Texto (fragmento)	Fecha
Ellaria Sand	indira_varma@gameofthron.es	Quidem nesciunt quam atque officia...	1985-02-24
Lisa Rasmussen	lisa_rasmussen@fakegmail.com	Illo nihil occaecati minima facere...	1976-12-18
John Roman	john_roman@fakegmail.com	Corporis soluta placeat porro quasi...	1987-08-19
Olly	brenock_o'connor@gameofthron.es	Dolor atque dolore ad...	1989-11-20
Alliser Thorne	owen_teale@gameofthron.es	Quos quia quos molestiae architecto...	2014-04-28

Se puede observar que algunos de los emails corresponden al dominio @gameofthron.es, lo que indica que parte de los datos de prueba fueron generados con nombres de personajes de la serie Game of Thrones. Los campos de texto contienen comentarios en latin (Lorem Ipsum), tipicos de datos de prueba generados automaticamente.

4.2 Peliculas obtenidas (coleccion: embedded_movies)

Se recuperaron 5 documentos de la coleccion embedded_movies. Esta coleccion es una version ampliada de la coleccion movies, ya que cada documento incluye un campo adicional llamado plot_embedding, que contiene un vector de cientos de valores de punto flotante. Estos vectores son representaciones numericas del resumen de la pelicula generadas por un modelo de lenguaje, y se utilizan para realizar busquedas semanticas.

Titulo	Genero	Anio	Calificacion IMDB	Duracion (min)
The Perils of Pauline	Action	1914	7.6	199
From Hand to Mouth	Comedy, Short, Action	1919	7.0	22
Scarface	Action, Crime, Drama	1932	7.8	93

The Charge of the Light Brigade	Action, Adventure, Romance	1936	7.1	115
Tarzan the Ape Man	Action, Adventure, Romance	1932	7.2	100

Todas las películas recuperadas son del género de acción y pertenecen a las primeras décadas del cine (1914-1936), lo que refleja que la base de datos tiene un catálogo amplio que incluye títulos clásicos. El campo `num_mflix_comments` indica cuántos comentarios tiene cada película en la plataforma, siendo Scarface la única con un comentario registrado.

5. Conclusiones

Esta practica fue muy util para entender como funciona la conexion entre una aplicacion Node.js y una base de datos MongoDB alojada en la nube. Aprender a usar el driver oficial de MongoDB permitio realizar operaciones de lectura sobre colecciones reales de manera sencilla, usando metodos como `find()`, `limit()` y `toArray()`.

Uno de los puntos mas importantes que se aprendio fue la necesidad de configurar los servidores DNS manualmente en el entorno de pruebas, ya que sin esa configuracion el programa no podia resolver la direccion del cluster de MongoDB Atlas y fallaba la conexion. Esto muestra que en el desarrollo de aplicaciones distribuidas, la configuracion de red es un aspecto critico que no se puede ignorar.

Tambien fue interesante explorar la coleccion `embedded_movies`, que contiene vectores de embedding en cada documento. Esto da una idea de como las bases de datos modernas no solo almacenan datos estructurados simples, sino tambien representaciones numericas generadas por inteligencia artificial que permiten realizar busquedas semanticas avanzadas, algo muy relevante para el desarrollo de aplicaciones como sistemas de recomendacion.

En general, la practica demostro que MongoDB es una herramienta muy flexible y potente para manejar grandes volumenes de datos en aplicaciones distribuidas, y que Node.js es un entorno adecuado para interactuar con ella gracias a su modelo asincrono basado en promesas.

6. Referencias

- MongoDB, Inc. (2024). MongoDB Node.js Driver Documentation.
<https://www.mongodb.com/docs/drivers/node/current/>
- MongoDB, Inc. (2024). MongoDB Atlas - Getting Started.
<https://www.mongodb.com/cloud/atlas>
- Node.js Foundation. (2024). DNS Module Documentation.
<https://nodejs.org/api/dns.html>
- MongoDB, Inc. (2024). Sample Mflix Dataset.
<https://www.mongodb.com/docs/atlas/sample-data/sample-mflix/>